AsanaAI: Explainable Deep Learning System for Yoga Pose Recognition

Shreyas Sawant 1023242
Amit Singh 1023254
Fr. C. Rodrigues Institute of Technology

April 18, 2026

Abstract

AsanaAI is a full-stack explainable deep-learning application for yoga pose classification from still images and live webcam input. The system uses two-stage transfer learning on MobileNetV2 trained with AdamW optimisation, MixUp regularisation, label smoothing, gradient clipping, and post-hoc temperature calibration. Predictions are made interpretable through Gradient-weighted Class Activation Mapping (Grad-CAM). The Streamlit interface supports dataset upload (ZIP), live-epoch training, comprehensive evaluation (accuracy, confusion matrix, per-class metrics, ROC/AUC), single-image inference, live webcam pose assistance, and pose guidance cards across 16 yoga classes. Trained checkpoints, class-name order, and calibration temperature are persisted and auto-loaded, removing the need for redundant retraining.

Keywords: Transfer learning, MobileNetV2, Grad-CAM, Streamlit, PyTorch, explainable AI, temperature calibration, MixUp, two-stage fine-tuning, webcam inference.

Contents

1 Introduction **3**

2 Dataset Description **3**
2.1 Class Set and Split . . . 3
2.2 EDA — Dataset Summary . . . 4

3 System Architecture and Design (Practical 4) **5**
3.1 End-to-End System Pipeline . . . 5
3.2 Model Architecture . . . 5
3.2.1 Backbone — MobileNetV2 . . . 5
3.2.2 Two-Stage Freezing Strategy . . . 5
3.2.3 Custom Classification Head . . . 5

4 Preprocessing, Augmentation and Training (Practical 4) **6**
4.1 Transform Pipelines . . . 6
4.2 MixUp Regularisation . . . 6
4.3 Training Hyperparameters . . . 7
4.4 Loss Function . . . 7
4.5 Post-hoc Temperature Calibration . . . 7
4.6 Training Results — Learning Curves . . . 8

5 Explainability — Grad-CAM (Practical 5) **8**
5.1 Mathematical Formulation . . . 8
5.2 Grad-CAM Pipeline . . . 9
5.3 Core Implementation . . . 9
5.4 Prediction Output . . . 10

6 Live Webcam Pose Assistance (Practical 5) **10**

7 Model Evaluation (Practical 5) **11**
7.1 Cached Inference Strategy . . . 11
7.2 Quantitative Metrics . . . 12
7.3 Evaluation Screenshots . . . 12
7.4 Qualitative Evaluation — Grad-CAM Inspection . . . 13

8 Engineering Highlights, Limitations and Conclusion **13**
8.1 Engineering Highlights . . . 13
8.2 Limitations . . . 14
8.3 Conclusion . . . 14

1 Introduction

Incorrect yoga posture is a common cause of preventable injury among beginners who practise without instructor supervision. An AI classifier that gives rapid pose identification and visual evidence of its reasoning can help fill this gap.

AsanaAI is built on three design pillars:

1. **Robust classification** — Two-stage transfer learning with MobileNetV2 achieves high accuracy on a dataset of 1920 images across 16 classes.
2. **Visual interpretability** — Grad-CAM overlays show which body regions drove each prediction, building user trust and enabling qualitative checks.
3. **Reproducibility and live access** — Checkpoint persistence, calibrated confidence, webcam input, and a Streamlit UI ensure consistent results across sessions.

2 Dataset Description

2.1 Class Set and Split

The dataset is a curated subset of publicly available yoga posture images, organised into 16 classes with a deterministic 80/20 physical train/val split.

Table 1: Dataset class summary (1920 images total, 96 train + 24 val per class)

Class	Sanskrit Name	Train	Val
Camel Pose	Ustrasana	96	24
Crane Pose	Bakasana	96	24
Eight-Angle Pose	Astavakrasana	96	24
Full Boat Pose	Navasana	96	24
King Dancer Pose	Natarajasana	96	24
Plow Pose	Halasana	96	24
Sitting Half Spinal Twist	Ardha Matsyendrasana	96	24
child_pose	Balasana	96	24
cobra	Bhujangasana	96	24
cow pose	Bitilasana	96	24
downward_dog	Adho Mukha Svanasana	96	24
plank	Phalakasana	96	24
seated_forward_bend	Paschimottanasana	96	24
tree	Vrikshasana	96	24
triangle	Trikonasana	96	24
warrior	Virabhadrasana I	96	24
Total		**1536**	**384**

The validation loader always uses `shuffle=False` to ensure deterministic, reproducible metric computation. Train and val images are stored in separate physical folder trees to prevent data leakage.

2.2 EDA — Dataset Summary

Figure 1: Screenshot: EDA Dashboard — Class Distribution & Sample Images

3 System Architecture and Design (Practical 4)

3.1 End-to-End System Pipeline

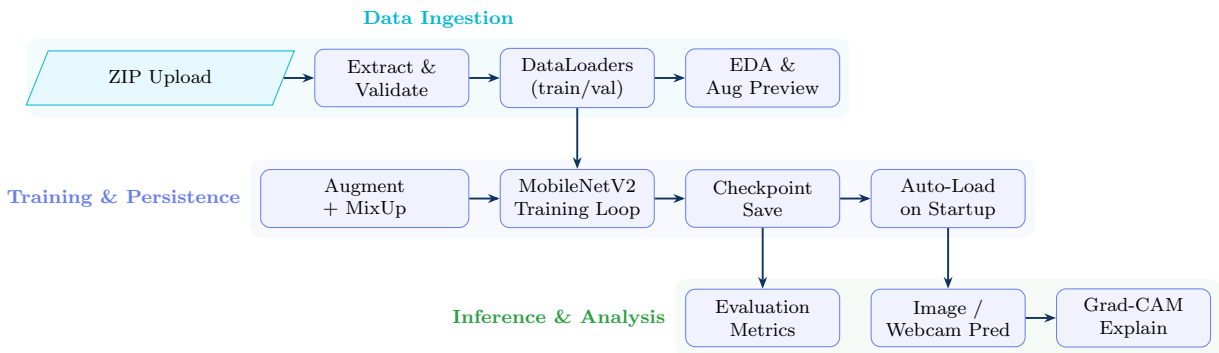


Figure 2: AsanaAI end-to-end system pipeline

3.2 Model Architecture

3.2.1 Backbone — MobileNetV2

MobileNetV2 uses inverted residual blocks with depthwise-separable convolutions, giving strong feature extraction at low computational cost, which makes it suitable for CPU-only lab machines. It is initialised with `MobileNet_V2_Weights.DEFAULT` (ImageNet pre-training).

3.2.2 Two-Stage Freezing Strategy

All backbone parameters are frozen during head warm-up (epochs 1–9). From epoch 10 onward, feature blocks at index ≥ 16 are selectively unfrozen with a lower learning rate (2×10^{-5}), allowing domain adaptation while keeping early spatial filters intact.

3.2.3 Custom Classification Head

The original MobileNetV2 classifier is replaced with:

$$\hat{y} = \text{Linear}_{128\to K}\Big(\text{Dropout}_{0.3}\big(\text{ReLU}\big(\text{Linear}_{1280\to 128}(\mathbf{z})\big)\big)\Big), \quad K = 16 \tag{1}$$

4 Preprocessing, Augmentation and Training (Practical 4)

4.1 Transform Pipelines

Table 2: Training vs. validation transform pipelines

Transform	Training	Validation
Resize / Crop	RandomResizedCrop(160, scale 0.85–1.0)	Resize(184) then CenterCrop(160)
Flip	RandomHorizontalFlip ($p = 0.5$)	None
Affine	Random ±10°, translate 4%, scale ±5%	None
Colour	ColorJitter (br/ct 0.12, sat 0.08, hue 0.02)	None
Sharpness	RandomAdjustSharpness(1.3 $p = 0.2$)	None
Normalise	ImageNet mean/std	ImageNet mean/std

Validation images are never augmented to ensure fair, unbiased evaluation.

4.2 MixUp Regularisation

For the first 10 epochs, MixUp augmentation blends pairs of training images to reduce overconfidence on the small dataset:

$$\tilde{x} = \lambda x_i + (1-\lambda)x_j, \quad \mathcal{L}_{\text{mix}} = \lambda\mathcal{L}(f(\tilde{x}), y_i) + (1-\lambda)\mathcal{L}(f(\tilde{x}), y_j), \quad \lambda \sim \text{Beta}(0.2,\, 0.2) \tag{2}$$

MixUp is disabled from epoch 11 so the fine-tuning phase trains on clean samples.

4.3 Training Hyperparameters

Table 3: Full training hyperparameter specification

Parameter	Value	Notes
Image size	160×160	Resize to 184, CenterCrop
Batch size	12	Balanced for CPU and GPU memory
Max epochs	24	With early stopping (patience = 6)
Optimizer (head)	AdamW, lr 5×10^{-4}	Classifier head only
Optimizer (fine-tune)	AdamW, lr 2×10^{-5}	Blocks ≥ 16 + head
Weight decay	10^{-4}	Both phases
LR scheduler	ReduceLROnPlateau	factor 0.5, patience 1, min 10^{-6}
Label smoothing	0.05	Applied in CrossEntropyLoss
Gradient clipping	1.0	Clips gradient norm
MixUp α	0.2	Epochs 1–10 only
Class weighting	Auto	Enabled if imbalance ratio ≥ 1.5
Fine-tune trigger	Epoch 10	Unfreeze blocks ≥ 16

4.4 Loss Function

Label-smoothed cross-entropy reduces overconfident predictions on the limited dataset:

$$\mathcal{L}_{\text{LS}} = (1-\varepsilon)\,\mathcal{L}_{\text{CE}}(p, y) + \frac{\varepsilon}{K}\sum_{k=1}^{K}\mathcal{L}_{\text{CE}}(p, k), \quad \varepsilon = 0.05,\ K = 16 \tag{3}$$

When the class imbalance ratio reaches ≥ 1.5, per-class weights $w_k = \bar{N}/N_k$ are applied automatically. An optional `WeightedRandomSampler` for oversampling minority classes is also available via a UI toggle.

4.5 Post-hoc Temperature Calibration

After training, a temperature scalar T is found on validation logits via grid search over $T \in [0.5, 2.5]$ (41 steps) to minimise negative log-likelihood:

$$T^* = \arg\min_{T}\ \frac{1}{N}\sum_{i=1}^{N} -\log\,\text{softmax}\Big(\frac{z_i}{T}\Big)_{y_i} \tag{4}$$

Calibrated probabilities at inference time are $p_k = \text{softmax}(z/T^*)_k$. The temperature value is saved in `training_history.json` and restored on load. For the trained model, the imbalance ratio was 8.15 (class weighting enabled), and the best validation accuracy reached **87.6%** at epoch 21.

4.6 Training Results — Learning Curves

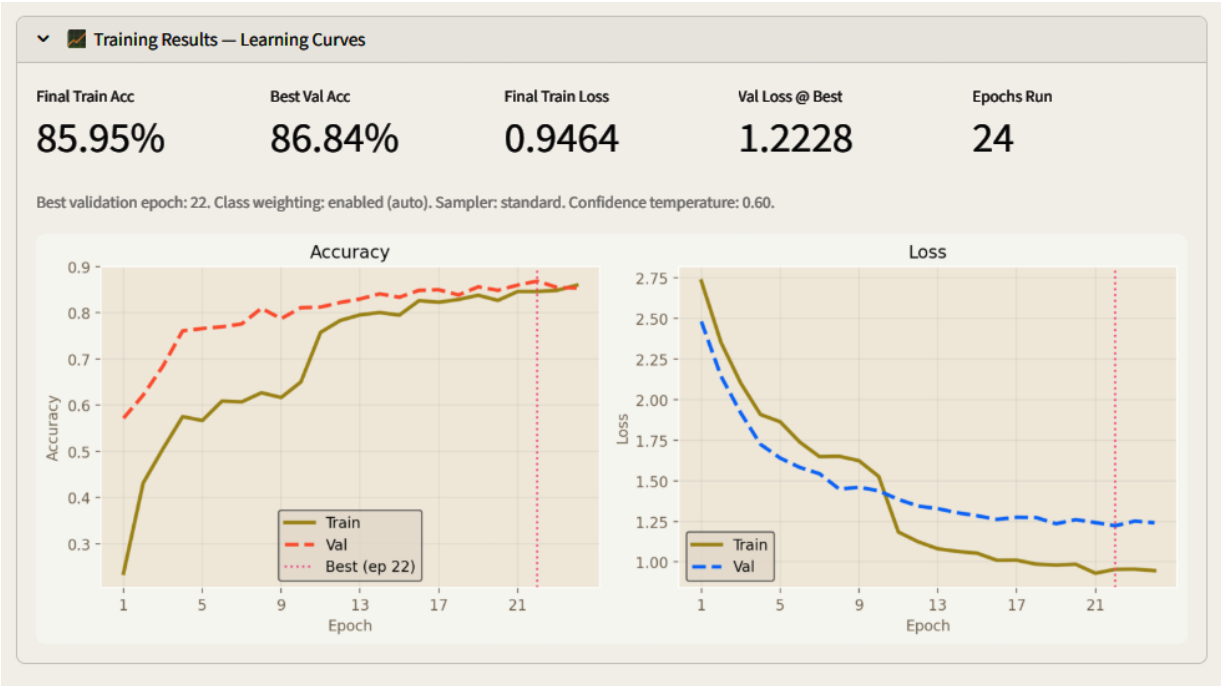


Figure 3: Screenshot: Training Learning Curves

5 Explainability — Grad-CAM (Practical 5)

Grad-CAM (Gradient-weighted Class Activation Mapping) provides spatial explanations without changing the network. It uses the gradient of the predicted class score with respect to the final convolutional layer's activation maps.

5.1 Mathematical Formulation

Let $A^k \in \mathbb{R}^{H' \times W'}$ be the k-th feature map of the last Conv2d layer and y^c the score for class c before softmax:

$$\alpha_k^c = \frac{1}{H'W'} \sum_{i,j} \frac{\partial y^c}{\partial A_{ij}^k} \qquad \text{(channel importance weight)} \tag{5}$$

$$L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_k \alpha_k^c A^k \right), \quad \text{normalised by } \max + 10^{-8} \tag{6}$$

The 10^{-8} guard prevents division by zero when activation maps are nearly flat. The result is resized to 160×160, coloured with the JET colourmap, and blended with the denormalised input at weights (0.6, 0.4).

5.2 Grad-CAM Pipeline

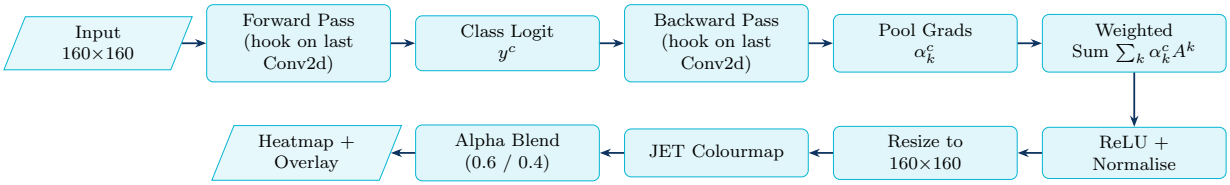


Figure 4: Grad-CAM computation pipeline as implemented in `generate_gradcam()`

5.3 Core Implementation

```python
def generate_gradcam(model, img_tensor, class_idx):
    activations, gradients = [], []
    target_layer = get_last_conv_layer(model)   # last Conv2d in features

    fwd = target_layer.register_forward_hook(
        lambda m, i, o: activations.append(o.detach()))
    bwd = target_layer.register_full_backward_hook(
        lambda m, gi, go: gradients.append(go[0].detach()))

    out = model(img_tensor)
    model.zero_grad()
    out[0, class_idx].backward()          # backward for predicted class
    fwd.remove(); bwd.remove()

    act, grad = activations[0][0], gradients[0][0]   # (C, H, W)
    weights = torch.mean(grad, dim=(1, 2))            # global avg pool
    cam = torch.relu(sum(w * a for w, a in zip(weights, act)))
    cam = cam / (cam.max() + 1e-8)                    # epsilon guard

    cam_r   = cv2.resize(cam.numpy(), (IMG_SIZE, IMG_SIZE))
    heatmap = cv2.applyColorMap(np.uint8(255*cam_r), cv2.COLORMAP_JET)
    overlay = cv2.addWeighted(orig, 0.6,
                  cv2.cvtColor(heatmap, cv2.COLOR_BGR2RGB), 0.4, 0)
    return cv2.cvtColor(heatmap, cv2.COLOR_BGR2RGB), overlay
```

Listing 1: Grad-CAM core logic (app.py)

5.4 Prediction Output

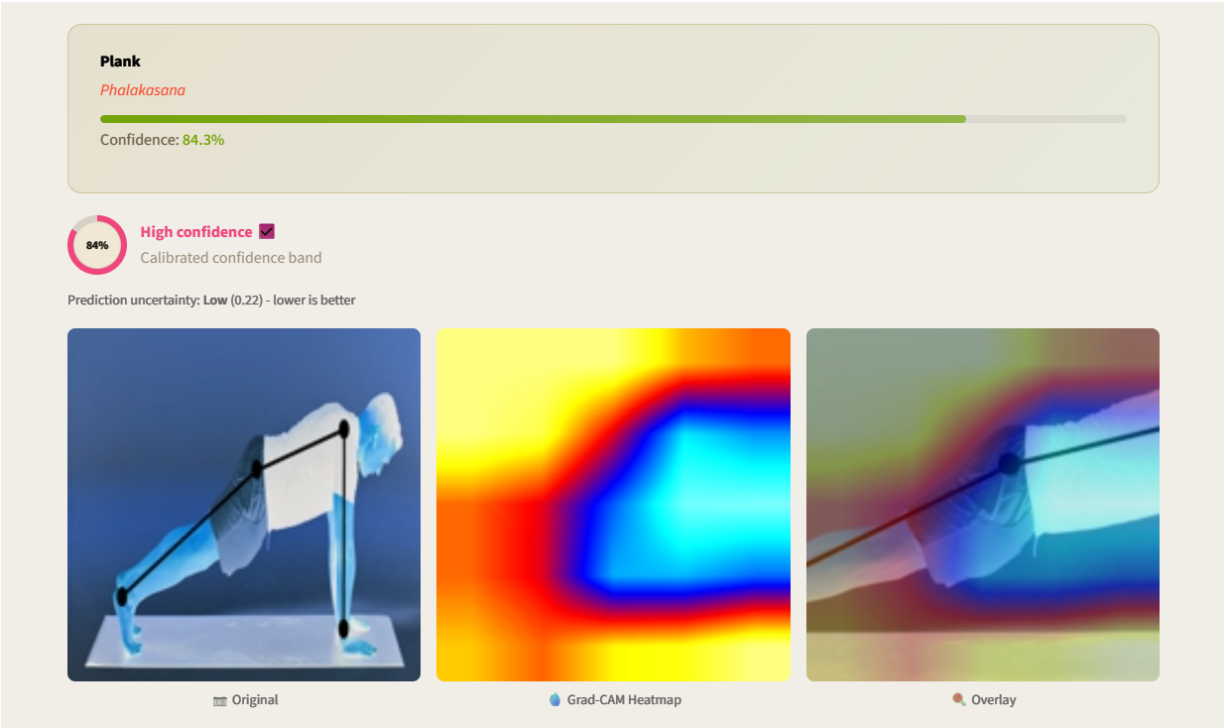


Figure 5: Screenshot: Prediction Panel with Grad-CAM Overlay

6 Live Webcam Pose Assistance (Practical 5)

In addition to single-image upload, AsanaAI provides a live webcam mode using Streamlit's built-in `st.camera_input`. Each captured frame is passed through the same inference pipeline (TTA-enabled, temperature-calibrated). Predictions across frames are smoothed using an exponential moving average (EMA) with $\alpha = 0.35$ to reduce flickering from minor pose variations:

$$\hat{p}_t = \alpha\, p_t + (1 - \alpha)\, \hat{p}_{t-1} \tag{7}$$

A confidence threshold of 0.45 is applied; frames below this threshold show a low-confidence warning instead of a pose name, preventing misleading feedback. The smoothed top-3 predictions and a live Grad-CAM overlay are displayed alongside the captured frame.

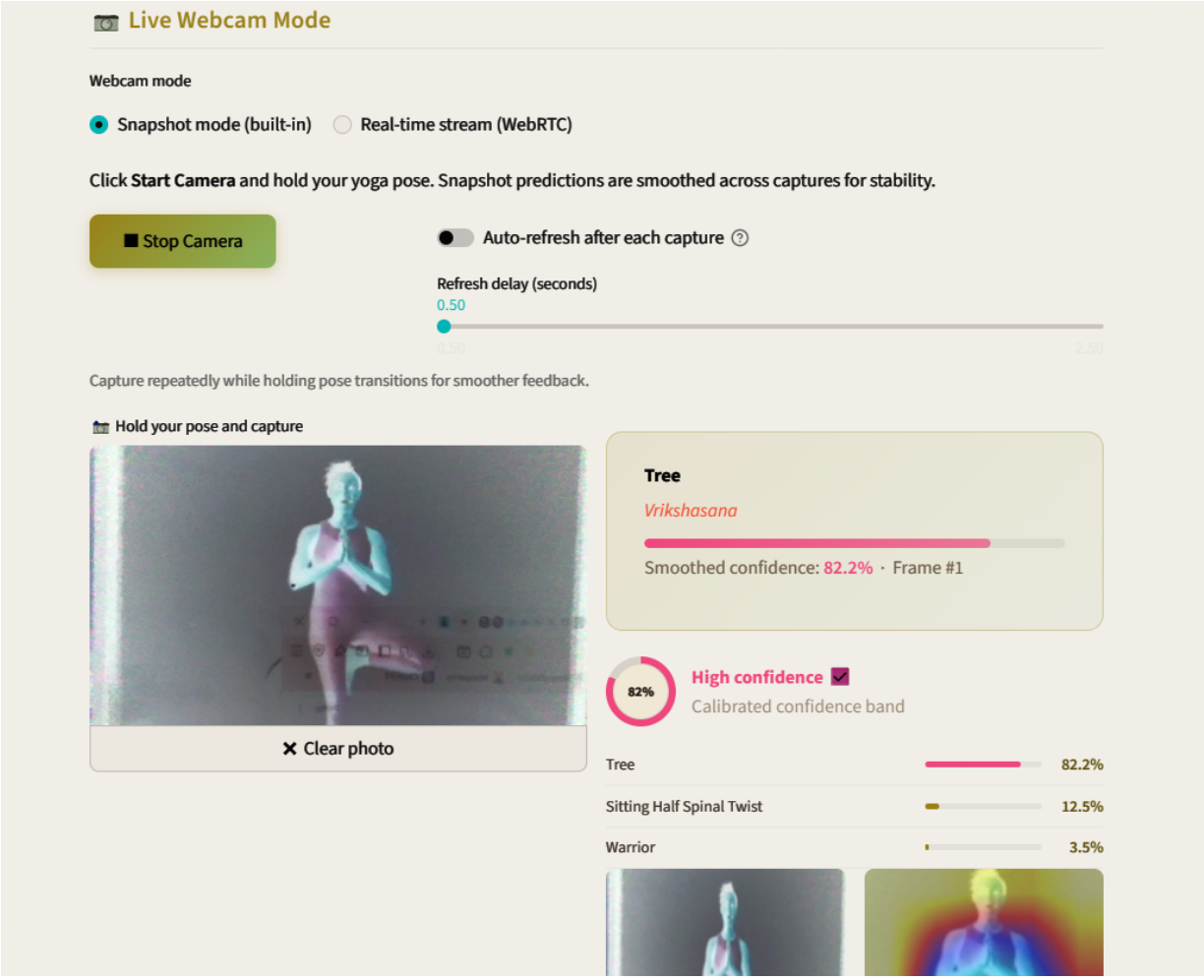


Figure 6: Screenshot: Live Webcam Mode

7 Model Evaluation (Practical 5)

7.1 Cached Inference Strategy

All validation predictions are computed once and cached via Streamlit's `@st.cache_data`, keyed on model ID, val-loader ID, and calibration temperature. The confusion matrix, classification report, per-class accuracy chart, and ROC curves all share this single cached inference pass, so switching tabs does not trigger redundant computation.

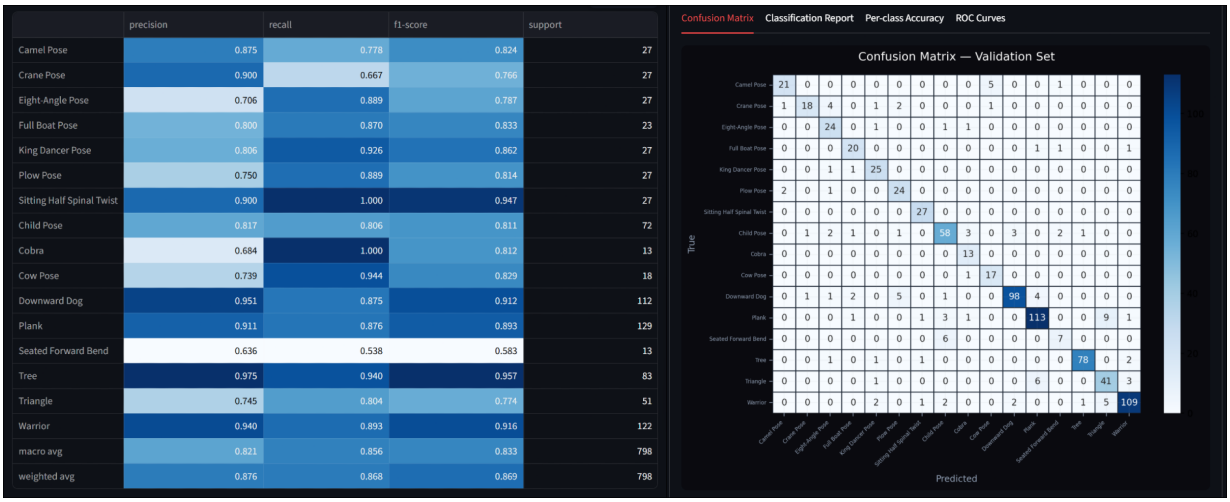
7.2 Quantitative Metrics

Table 4: Evaluation metrics computed over the 384-image validation set (16 classes)

Metric	Description
Overall accuracy	$(y_{\text{pred}} == y_{\text{true}})$.mean() over all 384 val images
Confusion matrix	16×16 heatmap of predicted vs. true class counts
Classification report	Per-class precision, recall, F1-score, support (scikit-learn)
Per-class accuracy	Colour-coded bars: green $\geq 80\%$, orange $\geq 60\%$, red $< 60\%$
ROC / AUC	One-vs-rest ROC curves with AUC for all 16 classes

For ROC curves, each class c uses its calibrated probability p_c as the score for a binary classifier (class c vs. all others). AUC close to 1.0 indicates strong separability. The trained model achieved a best validation accuracy of **87.6%** at epoch 21 out of 24 epochs.

7.3 Evaluation Screenshots



	precision	recall	f1-score	support
Camel Pose	0.875	0.778	0.824	27
Crane Pose	0.900	0.667	0.766	27
Eight-Angle Pose	0.706	0.889	0.787	27
Full Boat Pose	0.800	0.870	0.833	23
King Dancer Pose	0.806	0.926	0.862	27
Plow Pose	0.750	0.889	0.814	27
Sitting Half Spinal Twist	0.900	1.000	0.947	27
Child Pose	0.817	0.806	0.811	72
Cobra	0.684	1.000	0.812	13
Cow Pose	0.739	0.944	0.829	18
Downward Dog	0.951	0.875	0.912	112
Plank	0.911	0.876	0.893	129
Seated Forward Bend	0.636	0.538	0.583	13
Tree	0.975	0.940	0.957	83
Triangle	0.745	0.804	0.774	51
Warrior	0.940	0.893	0.916	122
macro avg	0.821	0.856	0.833	798
weighted avg	0.876	0.868	0.869	798

Figure 7: Screenshot: Confusion Matrix and Classification Report

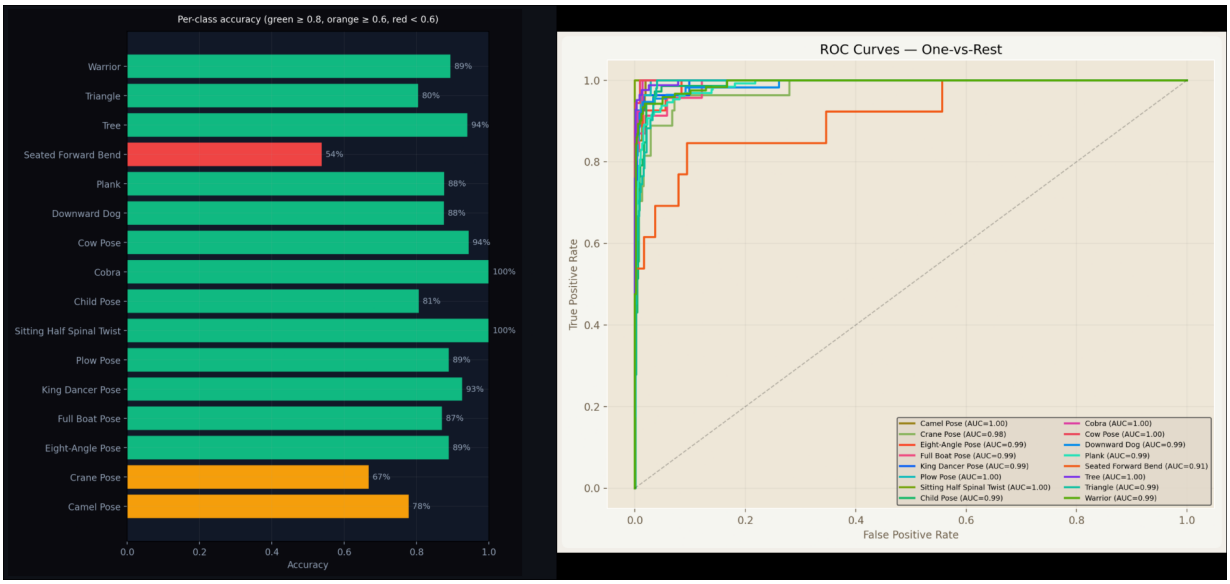


Figure 8: Screenshot: Per-Class Accuracy and ROC Curves

7.4 Qualitative Evaluation — Grad-CAM Inspection

Correct predictions highlight anatomically meaningful regions: the supporting leg for Tree pose, the extended arms for Warrior, the inverted hip for Plow, and the flat torso line for Plank. Misclassified samples typically show diffuse or background-focused activation, which is a useful diagnostic beyond accuracy numbers. Visually similar pose pairs (e.g., Crane vs. Eight-Angle) tend to have lower confidence scores, which the calibration step reflects accurately.

8 Engineering Highlights, Limitations and Conclusion

8.1 Engineering Highlights

E1. Two-stage transfer learning — Head warm-up then selective backbone fine-tuning improves accuracy and stability over end-to-end training from epoch 1.

E2. Deep-copy best state — `copy.deepcopy(model.state_dict())` captures the best-epoch weights, avoiding reference aliasing when training continues.

E3. Session-state guards — Upload and training callbacks are controlled by `dataset_loaded/trained` flags, preventing repeated side-effects.

E4. Cached evaluation inference — `@st.cache_data` with model and loader ID keys means all metric tabs share one validation forward pass.

E5. Temperature calibration — Post-hoc grid search gives well-calibrated confidence scores; the temperature value is saved with the checkpoint.

E6. Epsilon-guarded Grad-CAM — Normalisation uses `max + 1e-8` to handle near-zero activation maps without NaN or division errors.

E7. Palette PNG guard — `load_image_rgb()` converts P-mode images with byte-transparency via RGBA before RGB conversion.

E8. Webcam EMA smoothing — Exponential moving average across captured frames reduces flicker and gives more stable predictions during live use.

8.2 Limitations

L1. Still-frame classification cannot capture movement quality, pose transitions, or how a pose is entered and exited over time.

L2. Ninety-six training images per class limits diversity; accuracy may drop for unusual lighting conditions, body types, or camera angles.

L3. Evaluation tabs require the dataset ZIP to be re-uploaded when only a checkpoint is loaded (single-image and webcam prediction work from checkpoint alone).

L4. Without a CUDA GPU, 24-epoch training is noticeably slower; the app shows a prominent CPU-only warning at startup.

8.3 Conclusion

AsanaAI demonstrates a complete explainable-AI pipeline for yoga pose recognition across 16 classes, satisfying all Practical 4 (design and implementation) and Practical 5 (evaluation and explainability) objectives. The system combines two-stage MobileNetV2 transfer learning, MixUp and label-smoothing regularisation, post-hoc temperature calibration, epsilon-guarded Grad-CAM, live webcam inference with EMA smoothing, and a full evaluation suite — all within a session-state-driven Streamlit interface with checkpoint persistence and auto-reload.

Future improvements include: Optuna-based hyperparameter search, MediaPipe keypoint extraction for video-based posture correction, TorchScript/ONNX export for mobile deployment, and expanding the class set with a larger, more diverse dataset.